

Milestone 1 Report: Procedural Content Generation

Procedural Terrain Generator

Our group selected a Procedural Terrain Generator as our Procedural Content Generation project because it provides an effective way to apply mathematical concepts such as vectors, geometry and algorithms to solve a real-world problem. Rather than creating landscapes manually, procedural terrain generation uses mathematical equations to generate hills, valleys and other landforms automatically.

We chose this project because it demonstrates how complex environments can be created efficiently using computation while allowing us to explore the mathematical and programming principles behind procedural content generation. It also offers opportunities to develop a realistic and scalable terrain generation system that can be extended with additional features as the project progresses.

Group A members

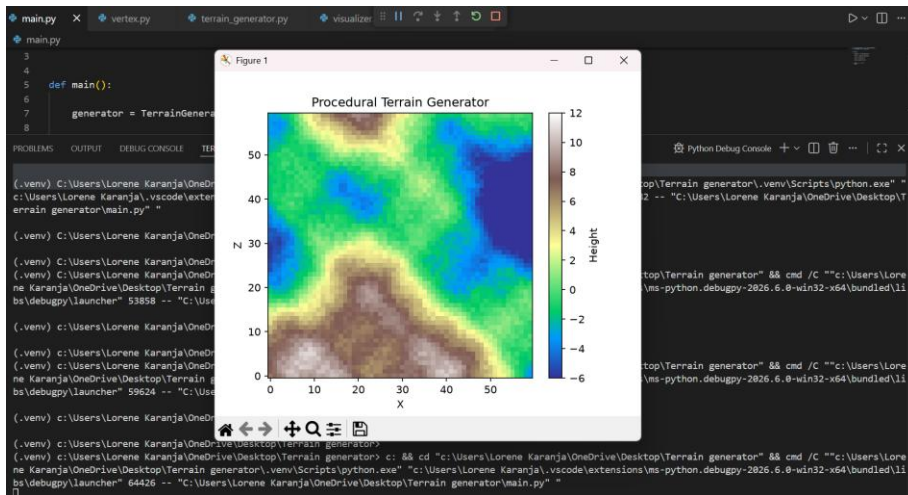
SCT 221-C004-0046/2024- Nasteha Ahmed
SCT221-C004-0581/2024- Lorene Wambui
SCT221-C004-0561/2024- Reyhana Abdi
SCT221-C004-0093/2024- Robert Bashir
SCT221-C004-0263/2024- Cecilia Wambui

1. Introduction

The aim of this milestone is to build the foundation of a procedural terrain generator. Instead of relying on existing terrain-generation libraries, the system generates terrain using mathematical equations. The terrain is represented as a collection of three-dimensional vertices and visualised as a height map.

2. Working Prototype

The prototype successfully generates a terrain grid where each point is represented by a Vertex (x, y, z) . The x and z coordinates define the horizontal position while the y coordinate represents elevation. Terrain heights are calculated using layered sine and cosine functions with a small amount of random variation to produce natural-looking hills and valleys. The generated terrain is displayed using Matplotlib with a terrain colour map.



```
1 import math
2 import random
3 from vertex import Vertex
4
5 class TerrainGenerator:
6
7     def __init__(self, width=60, depth=60):
8         self.width = width
9         self.depth = depth
10        random.seed(42)
11
12    def generate(self):
13        terrain = []
14
15        for z in range(self.depth):
16            row = []
17
18            for x in range(self.width):
19
20                # Large hills and valleys
21                large = (
22                    math.sin(x * 0.07) * 5 +
23                    math.cos(z * 0.08) * 5
24                )
25
26                # Medium hills
27                medium = (
28                    math.sin((x + z) * 0.18) * 2 +
29                    math.cos((x - z) * 0.15) * 1.5
30                )
31
32                # Small surface details
33                small = (
34                    math.sin(x * 0.5) *
35                    math.cos(z * 0.5) * 0.8
36                )
37
38                # Small random variation
39                noise = random.uniform(-0.5, 0.5)
40
41                # Final terrain height
42                y = large + medium + small + noise
43
44                row.append(Vertex(x, y, z))
45
46            terrain.append(row)
47
48        return terrain
```

```
1 import matplotlib.pyplot as plt
2 class Visualizer:
3
4     def display(self, terrain):
5
6         heights = []
7
8         for row in terrain:
9
10            height_row = []
11
12            for vertex in row:
13                height_row.append(vertex.y)
14
15            heights.append(height_row)
16
17        plt.figure(figsize=(8, 8))
18
19        plt.imshow(
20            heights,
21            cmap="terrain",
22            origin="lower",
23            vmin=-6,
24            vmax=12
25        )
26
27        plt.title("Procedural Terrain Generator")
28        plt.xlabel("x")
29        plt.ylabel("z")
30        plt.colorbar(label="Height")
31        plt.show()
32
33 main.py
34
35 1 from terrain_generator import TerrainGenerator
36 2 from visualizer import Visualizer
37
38 3
39 4
40 5 def main():
41
42     6 generator = TerrainGenerator()
43
44     7
45     8 terrain = generator.generate()
46
47     9
48     10 viewer = Visualizer()
49
50     11
51     12 viewer.display(terrain)
52
53 13
54 14
55 15
56 16 if __name__ == "__main__":
57     17 main()
```

```
1 class Vertex:
2
3     def __init__(self, x, y, z):
4         self.x = x
5         self.y = y
6         self.z = z
```

3. Mathematical Derivation of the Representation

The terrain is represented as a two-dimensional grid of vertices. Each vertex is defined by:

x – horizontal position
y – terrain height (elevation)
z – depth position

Each vertex is represented mathematically as:

$$V = (x, y, z)$$

The terrain height (y) is calculated by combining four mathematical components:

Large Hills and Valleys

$$\text{large} = \sin(x \times 0.07) \times 5 + \cos(z \times 0.08) \times 5$$

This equation creates the overall shape of the terrain by generating broad hills and valleys.

Medium Hills

$$\text{medium} = \sin((x + z) \times 0.18) \times 2 + \cos((x - z) \times 0.15) \times 1.5$$

This layer adds medium-sized hills between the larger landforms.

Small Surface Details

$$\text{small} = \sin(x \times 0.5) \times \cos(z \times 0.5) \times 0.8$$

This equation introduces smaller bumps and surface irregularities to make the terrain appear more natural.

Random Variation

$$\text{noise} = \text{Random}(-0.5, 0.5)$$

A small random value is added to reduce repetitive patterns and introduce natural variation.

Final Height Equation

$$y = \text{large} + \text{medium} + \text{small} + \text{noise}$$

The height y is measured in relative units. These values can be scaled to real-world measurements if required.

If 1 unit = 100 meters, then a height of 30 represents an elevation of approximately 3000 meters above sea level.

4. Core System Representation

The system consists of four main components:

main.py – Controls the execution of the program by creating the terrain generator and visualizer.

terrain_generator.py – Calculates the height of every vertex using mathematical equations.

vertex.py – Defines the data structure used to store the coordinates of each terrain point.

visualizer.py – Displays the generated terrain as a height map using Matplotlib.

5. System Design Explanation

Main Program



Terrain Generator



Generate Vertices



Store Terrain Grid



Visualizer



Terrain Height Map

